DocPilot: Architecture Comparison Report

Modular Monolith vs. Microservices — Based on Actual Benchmark Data

> **Methodology:** `scratch_benchmark.py` was executed locally.
>
> - **Monolith** = direct Python in-process function call (DocumentRepository accessed in memory)
> - **Microservice** = HTTP GET call over TCP loopback to an embedded FastAPI server (identical data, full serialize/deserialize cycle)
> - **Iterations:** 100 per mode | **Concurrency:** 10 workers | **Platform:** Windows, Python 3.12

NFR 1: Response Time (Latency)

Metric	Monolith (ms)	HTTP / Microservice (ms)	Delta (ms)
Mean	0.0026	2.0605	**+2.0579**
Median	0.0025	2.0134	**+2.0109**
Std Dev	0.0005	0.4122	—
Min	0.0024	1.4034	—
Max	0.0065	4.1317	—

Finding: The HTTP boundary adds a **mean overhead of 2.06 ms** per inter-service call. In this benchmark, the in-process call exhibits **~792× lower latency** than an HTTP-based call due to the absence of network serialization and transport overhead. The microservice P99 reaches 3.14 ms — compared to 0.0049 ms for the in-process call.

> [!NOTE]
> The 2 ms overhead shown here represents **one** service boundary. In multi-hop microservice pipelines (e.g., API Gateway → Query Svc → Search Svc → DB

> Svc), this overhead can accumulate to an **estimated 6–10 ms** of pure architectural overhead per request — before LLM generation even begins.

NFR 2: Throughput (Requests Per Second under Concurrency)

Pattern	RPS (10 workers, 50 calls)
Monolith (in-process)	**7,285.87 RPS**
HTTP / Microservice	**5.49 RPS**
Gain Factor	**1,327x**

Finding: Under 10 concurrent workers, the monolith achieves **7,285 RPS** vs only **5.49 RPS** for the HTTP path. The observed throughput difference is influenced by HTTP overhead, connection handling, and the limitations of the local benchmarking setup (single-node execution, no connection pooling, no horizontal scaling). In a distributed deployment, microservices can scale horizontally, mitigating this gap.

> [!IMPORTANT]
> The raw throughput gap (1327x) is an upper bound specific to this local, single-node setup. It should not be interpreted as a universal property of either architecture. In production, DB I/O equalises the gap considerably, and microservices gain back throughput through independent horizontal scaling.

Trade-off Analysis

Why Modular Monolith Wins for DocPilot Now

Dimension	Monolith	Microservices
Latency	0.003 ms internal hops	2+ ms per service boundary
Throughput	7,285 RPS (bounded by CPU/DB)	5.49 RPS (bounded by HTTP overhead)
Deployment	1 container	4+ containers + orchestration
Debugging	Single stack trace	Distributed tracing required

Dimension	Monolith	Microservices
Dev Velocity	Single codebase, no contracts	Service contracts (OpenAPI/protobuf)

When Microservices Become Worth It

1. **Scaling a specific bottleneck independently:** If the Embedding Service needs 20 GPU workers but the API needs only 2 replicas, microservices allow paying only for what you need.
2. **Team isolation:** If >10 developers work in parallel, service ownership boundaries prevent merge conflicts.
3. **Technology heterogeneity:** If the Search Service needs Go for low-latency and the Ingestion Service needs Python for ML libraries, microservices allow mixing stacks.

Conclusion

The benchmark demonstrates that a modular monolith eliminates approximately **2 ms of latency per service boundary** compared to HTTP-based microservices in a single-node setup. While the monolith shows significantly higher throughput in this configuration, this advantage is partly due to the absence of network overhead and the limitations of the local benchmarking environment. Microservices, however, enable **horizontal scaling and independent service deployment**, which can offset these costs at larger scales. For DocPilot's current workload, the modular monolith provides superior latency and simplicity, while preserving a clear path to future decomposition via its clean `api / services / infrastructure` layering.